SQLite Optimization Migration Issues

Problem Description

When running consecutive `migrate:fresh` operations (especially `migrate:fresh` followed immediately by `migrate:fresh --seed`), you may encounter the error:

```
SQLSTATE[HY000]: General error: 11 database disk image is malformed
```

This issue is commonly caused by the SQLite optimization migration that switches the database to WAL (Write-Ahead Logging) mode.

Root Cause Analysis

The migration `001_optimize_sqlite_configuration.php` applies several SQLite PRAGMA statements that can interfere with rapid database recreation:

Key PRAGMA Settings and Their Impact:

1. `journal_mode = WAL` (Write-Ahead Logging)
   - **Critical**: Changes SQLite from default DELETE mode to WAL mode
   - **Impact**: Creates additional files (`database.sqlite-wal`, `database.sqlite-shm`)
   - **Problem**: These files can persist even after deleting the main database file
2. `synchronous = NORMAL`
   - Reduces disk sync frequency (vs FULL)
   - Can leave transactions in inconsistent state if interrupted
3. `busy_timeout = 5000`
   - Sets 5-second timeout for locked database operations
   - Could cause delays if database is locked
4. `mmap_size = 2147483648` (2GB memory mapping)
   - Uses memory mapping for large database access
   - Could affect file locking behavior
5. `auto_vacuum = incremental` & `incremental_vacuum`
   - Changes how SQLite manages freed space
   - Could affect database file structure

Why This Causes Migration Issues:

When you run `migrate:fresh` followed immediately by `migrate:fresh --seed`:

1. **First `migrate:fresh`**:
   - Drops tables and recreates them
   - Runs the optimization migration, switching to WAL mode
   - Creates `.sqlite-wal` and `.sqlite-shm` files
2. **Second `migrate:fresh --seed`**:

- Tries to drop all tables again
- **Problem**: WAL mode files might still be in use or corrupted
- The "database disk image is malformed" error occurs because:
  - WAL files are inconsistent with the main database
  - File system locks from previous operation still active
  - Memory-mapped regions not properly released

Solutions

1. Proper Cleanup Between Operations

```
# Close all database connections and clean up WAL files
php artisan db:wipe
rm -f database/database.sqlite*
touch database/database.sqlite
php artisan migrate:fresh --seed
```

2. Thorough Database Reset

```
# Kill any processes that might be using the database
sudo lsof +D database/ 2>/dev/null || true

# Remove all database files completely
rm -f database/database.sqlite*

# Clear Laravel caches that might hold database connections
php artisan cache:clear
php artisan config:clear
php artisan route:clear
php artisan view:clear

# Create fresh database file
touch database/database.sqlite

# Set proper permissions
chmod 664 database/database.sqlite

# Run migration with seeding
php artisan migrate:fresh --seed
```

3. Modify Optimization Migration for Development

Update the optimization migration to be less aggressive during development:

```
public function up(): void
{
    $pragmas = [
```

```
            'auto_vacuum = incremental',
            'busy_timeout = 5000',
            'cache_size = -64000',
            'foreign_keys = ON',
            'incremental_vacuum',
            'mmap_size = 2147483648',
            'page_size = 32768',
            'synchronous = NORMAL',
            'temp_store = MEMORY',
            'wal_autocheckpoint = 1000',
        ];

        // Only apply WAL mode in production
        if (app()->isProduction()) {
            $pragmas[] = 'journal_mode = WAL';
        } else {
            $pragmas[] = 'journal_mode = DELETE';
        }

        foreach ($pragmas as $pragma) {
            DB::statement("PRAGMA {$pragma}");
        }
    }
```

4. Add Delay Between Operations

```
php artisan migrate:fresh
sleep 1
php artisan migrate:fresh --seed
```

Prevention

- Avoid running multiple `migrate:fresh` commands in rapid succession
- Use `migrate:fresh --seed` in a single command when possible
- Consider disabling WAL mode during development
- Always clean up WAL files when manually resetting the database

Notes

- WAL mode is excellent for production environments with concurrent access
- The issue primarily affects development workflows with frequent database resets
- WAL files (`.sqlite-wal`, `.sqlite-shm`) are automatically created and managed by SQLite
- These optimization settings significantly improve SQLite performance in production